# Exercise 2

## Introducing Topics and Messages
### (and the Run Loop)

In this exercise, we will create a `Topic` that our `actuator` publishes to and that the `controller` is subscribed to. By the end, you will also understand the general function of a node's run loop and how to adapt it to your needs.

## Task 2.1: Understanding Topics & Messages

While nodes can send commands to each other via queues, there is a second way of passing along information: topics.

Topics provide a way of distributing information among nodes. One node, the publisher, sends messages to a topic, e.g. the current position of the robot, which are then stored in the topic's buffer. The messages in this buffer buffer can then be retrieved and read by any node that subscribes to the topic.

**Queues vs. Topics**

There are a couple important differences between queues and topics that will hopefully give you an idea of when to use which.

First, and foremost at the most basic level, queues and topics are the inverse of each other. A queue has one endpoint (the node it is associated with), but an unlimited number of nodes can put information into the queue. A topic, on the other hand has exactly one publisher that can put information into the topic, but an unlimited number of nodes can access that information once it is published.

Another important difference is that queues get emptied as a node works through the commands sent there, but topics keep a more permanent record of the messages published to them. When a subscriber node "retrieves" a message, it collects a copy of the data stored to the topic's buffer. It can't alter or delete any information that has been published to the buffer.

As a result of these differences (and a few technical considerations surrounding overhead, multiprocessing, etc. that you can read about in the documentation), queues should primarily be used to send function commands between nodes. Any time you want to pass larger pieces of data between nodes or you are handling data that more than one recipient would be interested in, it makes more sense to publish it to a topic instead.

**Messages & Topic Config**

In order for topics to function, we need to clearly define the shape and format of the messages that can get published to each of them. For example, a topic `robot_pose` might hold messages that contain the robot's `x` and `y` position (its "fields"). For our robot, these are both floating point numbers (`np.dtype("float64")`), so we need to define somewhere that the message fields `x` and `y` in topic `robot_pose` will always be two floats. This happens in another file in the `configs` folder: `configs/topic_config.yaml`. where you would find an entry looking something like this:

```
robot_pose:
  buffer_length: short
  dtypes:
    names: ["timestamp", "x", "y"]
    formats: ["int", "float32", "float32"]
```

The topic is identified by a unique name (here `"robot_pose"`) and has a dictionary of meta-information associated with it.

For each field in your message, there is a field name and a numpy data type defined in the corresponding lists of the `dtypes` sub-dictionary. In this example, the message has three fields: an integer `timestamp` and our floats `x` and `y`. (You can find a full list of supported numpy dtypes at the top of the `topic_config` file.)

When you publish a message, you don't need to manually set the timestamp, that happens automatically inside the publishing function where the node adds a the output from `time.time_ns()`. The only thing you need to do is to make sure that in the config, the first field of a message is always defined as `"timestamp"` with the format `"int"`. (If you want to know why we do not use `time.time()`, you can read up on it in the documentation.)

To give you another example of what a topic might look like, imagine the `actuator` you created in the last exercise wanted to publish the result of their print and math functions to one topic. All messages published to one topic need to have the same format, so you would need one field for the strings printed in `PRINT` mode, and one for the product calculated in `MATH` mode. When publishing to the topic, the `actuator` would simply leave one of the two message fields blank, depending on which mode it was in.

(Yes, in reality you would probably use two topics for this, but go with it for now.)

The config for a topic like this might be defined as follows:

```
print_log:
  buffer_length: long
  dtypes:
    names: ["timestamp", "print_string", "product"]
    formats: ["int", "U40", "float32"]
```

`U40` is the numpy name for a Unicode string with a length of 40 characters. As we said above, all messages need to have a consistent, fixed length, so you have to define a maximum length for fields of type string. Shorter strings are no problem and will be handled by python in the background, but if you try to publish one that is longer than your topic config allows, you will get an error.

The other element that you will find in the topic config is the `buffer_length` (often shortened to `bl`). Topics function as a ring buffer, meaning that they have an array of a specified length `bl`, in which they store any messages published to them. The array is filled up as messages come in, and once the final slot has been filled, the topic will go back to the beginning of the array and start overwriting the oldest messages.

There are currently four buffer lengths: `"min"`, `"short"`, `"medium"`, and `"long"`. You can set their numerical value at the top of the `topic_config` file and the `config_parser` will automatically replace the placeholder strings when you start up the framework. Choosing the right buffer length is a balance between your implementation's need to access historic data and the memory required to store each message. (You can find more information on that in the note at the end).

**Your Task:**

- Add a topic to the `topic_config`, using the template provided at the top of the file.
- Decide on at least 3 message fields (in addition to `timestamp`) and add their names and formats to the `dtypes` section.
- Decide on an appropriate buffer length and add that, too.

## Task 2.2: Publishing a Message

Once we know what our messages look like, we need to write a function that actually publishes them to their topic. In order for a node to be able to send messages to a topic, it needs to be the designated publisher. Every topic can have only one publisher, though a single node can be the publisher for several different topics.

If you've read through the information at the top of the topic config file, you will have seen that there are three elements that get added to each topic's config during parsing: `publisher`, `subscriber`, and `lock`. This means that once the framework is running, you will be able to access the name of the topic's publisher and a list of its subscribers through the topic config (these also get stored as attributes, see the technical reference for details). However, when you are writing the configs, you don't add the names of the publisher and subscriber nodes to the topic config file. Instead, this is defined in the node config.

As you may recall from exercise 1, every node config has two lists: `pubs` and `subs`. One contains the names of the topics that node publishes to, the other the topics the node subscribes to:

```yaml
  vision:
    class_name: VisionNode
    queues: ["camera"]
    topics:
      pubs: ["marker_detections"]
      subs: ["camera_capture", "robot_pose"]
```

During parsing, when the framework initializes, the information about publishers and subscribers is extracted from the node config and added to the topic configs. It is done this way so you only have to track the relationships between nodes and topics in one place, rather than having to update two separate configs every time you add a subscriber, for example.

If you try to assign to publishers for a single topic (i.e. add the topic's name to the `subs` list of two separate nodes), you will get an error and the framework will exit before starting up. Similarly, you will get a warning if a topic has nodes subscribed to it but no assigned publisher, though in this case, the framework will still run, the subscribers simply won't be able to read anything from the topic. If a node has neither a publisher nor a subscriber, it is not going to be included in the framework.

The method that publishes a message to a topic is called, unsurprisingly, `publish_to_topic()`. It takes as parameters the name of the topic `(str)`, the message to be published (in the form of a `list` or a `tuple`), and two booleans, `pad_arrays` and `casting`.

For our hypothetical `robot_pose` topic from earlier, an implementation might look something like this:

```
def move_robot(self):
    # move functionality happens here
    x, y = self.get_new_position() # returns two floats
    self.publish_to_topic(
        topic_name="robot_pose",
        msg=(x,y)
    )
```

`pad_arrays` and `casting` both have default values, so they can be omitted in the function call. We will discuss `pad_arrays` in greater detail in Task 2.5, but `casting` (default: `True`) tells the function whether it can apply `numpy`'s safe casting when writing a message to the buffer. For example, with `casting=True`, you could publish the message `(2,3)` (i.e. `x=2, y=3`) to the `robot_pose` topic and they would be silently turned into floats `(2.0,3.0)`. If you set `casting=False`, trying to publish `(2,3)` would cause an error. (For more details on what is and is not considered `casting`, see the documentation.)

**Your Task:**

- Select (or build) a publisher node for your topic and put the topic's name in that node's list of `pubs` in the `node_config`.
- Write a function for your node that creates a properly formatted message (i.e. matching the format you defined in Task 2.1) and send it to the topic by using `publish_to_topic()`.
- Add a key press trigger for your function to the menu.

If you want, you can run your code now and press the chosen key to publish the message. This will tell you if you've gotten the message formating right because there will be an error or a warning if you haven't, but you're not going to see any effect yet. For that, we need:

## Task 2.3: Reading from a Topic

Now, we need to choose (or build) a node that subscribes to the new topic. This can be the same node that is also publishing to the topic or a different node (or an arbitrary number of different nodes).

A node uses the method `read_topic()` to retrieve messages from a topic it's subscribed to. There are two different ways a subscriber can read messages from a topic, which are determined by the arguments you pass to the method:

- `max_msgs=m, new_only=True`: Calling the method with `new_only=True` means the node will only retrieve messages that have been published since this subscriber last read the topic. Each subscriber node individually tracks which messages it has and hasn't read, so reading new messages with one subscriber won't mark them as read for another subscriber.

  The `max_msgs` parameter allows you to set a maximum of new messages you want to read. If 5 new messages were published, but you set $m = 3$, you will only get the 3 newest messages (though the node will consider all of the messages as read). If $m = 3$ but there are only 2 new ones, these 2 will be returned. If you set `max_msgs=0`, you will get every message that was published since the subscriber last checked the topic.

- `max_msgs=n, new_only=False`: Setting `new_only` to `False` gets you the newest $n$ messages, no matter if you have seen them before or not. If there are fewer than $n$ messages published to the topic so far, you will get all of those messages or `None` if there are none.

  Two things are important to note here. First: if `new_only` is `False`, `max_msgs` *cannot* be 0! That would cause a warning and nothing would be returned. And second: just like the first variant, this call to `read_topic()` will mark all messages that are currently published to the topic as "read".

Let's look at some pseudo-code to illustrate the function of these parameters:

```
publisher.publish_to_topic(topic_name="test", msg=("Test1",))
publisher.publish_to_topic(topic_name="test", msg=("Test2",))

subscriber.read_topic(
    topic_name="test",
    max_msgs=1,
    new_only=False) # get 1 newest message whether seen before or not
# -> returns message "Test2", marks both messages as read

publisher.publish_to_topic(topic_name="test", msg=("Test3",))
subscriber.read_topic(
    topic_name="test",
    max_msgs=0,
    new_only=True) # get all unread messages
# -> returns only message "Test3", since older messages are marked as read

subscriber.read_topic(
    topic_name="test",
    max_msgs=2,
    new_only=False)
# -> returns messages "Test2" and "Test3"

subscriber.read_topic(
    topic_name="test",
    max_msgs=0,
    new_only=True)
# -> returns None
```

*Note: `publisher.` and `subscriber.` are used as shorthand here to illustrate the point. In reality, these function calls would probably happen inside separate nodes, they would be called using `self.read_topic()`, and the messages would be returned in lists (cf. note below).*

When building your robot implementation, be sure you are intentional about which of the two reading-modes you choose. For example: if you have a vision node that analyzes camera images and does marker detection based on the robot's current position, you would probably want to implement it something like this:

```python
def detect_markers(self):
    # get latest camera image
    img_msgs = self.read_topic(
        topic_name="camera_capture",
        max_msgs=1,
        new_only=True)
    if img_msgs is None:
        # no new image, so can't run detection
        return
    img_msg = img_msgs[0] # read_topic returns a list. See note below
    img = img_msg[2] # the image is the second message field (cf. topic config)

    # get the current robot pose
    robot_pose_msgs = self.read_topic(
        topic_name="robot_pose",
        max_msgs=1,
        new_only=False)
    if robot_pose_msgs is None:
          return
    # unpack message (cf. topic config)
    timestamp, xr, xy, theta, _ = robot_pose_msgs[0]
```

You only want to run the marker detection algorithm if the camera has published a new image, otherwise, it would be wasted effort, so when reading the topic `camera_capture`, you choose `new_only=True`.

However, the `robot_pose` topic only gets updated if the pose has changed. Imagine your robot is standing still, but the markers it sees are moving, so you still want to keep running marker detection (of course, this will not happen in the exam, but trust us, you *will* be moving markers around while you test out the vision node).

If you call `read_topic()` for the `robot_pose` topic and set `new_only=True`, the return value would be `None`, since the robot has not moved, and the function would exit here. To avoid this, you need to set `new_only=False`, so you're always getting the current robot pose, even if there hasn't been an update in a while because the robot has been standing still.

*(For the question of how to make sure that the camera image and robot pose you run the detection on are in sync when the robot is moving and there* are *new messages published to `robot_pose`, see the documentation section about retrieving synced messages.)*

**Important Note:** `read_topic()` always returns a **list** of messages, even if that list contains only a single element. The only exception is if no messages were retrieved or there was an error, in which case it returns `None`. If you are having trouble parsing the messages your subscriber node read from a topic, maybe double-check whether you unpacked the list of return messages before trying to access specific message fields.

*(This and other helpful tips are also included in the Tips and Troubleshooting section of the documentation.)*

**Your Task:**

- Choose (or build) a subscriber node for your topic and add the topic name to the node config's `subs`-list.
- Write a function (e.g. `print_topic`) for your subscriber node that can read new messages from your topic and print them to the console (or do other fun things with them).
- Add a key press trigger for that function to the menu.
- Run your code and publish and read messages using keystrokes. Vary the arguments you give to the `read_topic()` method to see how they affect the outcome.

## Task 2.4: The Run Loop

In many cases, tasks like publishing to or reading from a topic shouldn't just happen a single time when triggered by a specific command or key press. Instead, you will probably want your nodes to publish messages or check a topic for new messages at regular intervals. For this, we will have a look at the run loop that administers the operation of our nodes.

Every node has an internal loop that is executed when the node registry calls `node.start()` (inside the main process's `Main.run()`). It looks approximately like this:

```
def run(self):
    self.runnning = True
    self.start_time = time()
    self.next_step = self.start_time

    while self.running:
    # calculate how long iteration of run loop should take
    self.next_step += self.dt

    # check system queue and run all commands inside
    self.check_queue(max_msgs="all", sys_q=True)
    # get one command from front of command queue and run it
    self.check_queue(max_msgs=1)

    # execute all of the node-specific logic
    self.run_step()

    # calculate time remaining in loop iteration and either wait
    # until dt is reached or give a warning.
    [Code omitted here! See Node.run() inside "node.py" for details.]

    self.count += 1

# orderly shut-down once 'self.running' was set to false
# (e.g. because stop key was pressed)
self.shut_down()
```

*(Some function names and parameters are abbreviated or altered for summary purposes. Check `node.py` or the technical reference to see the accurate method signatures.)*

So, the steps that happen during each iteration of the run loop are:

1. **Check system queue:**
   - Check if there have been commands sent from the `main` node and execute them.

- These are usually administrative commands like `switch_mode()`, `flush_queue()`, or `stop()`, but also include commands triggered by keyboard inputs.
- All commands inside the system queue are retrieved and worked through in their entirety (order: first-in-first-out) during every iteration of the run-loop.

2. **Check command queue**:
   - Check if any commands have been sent by other nodes to this node's command queue and execute them.
   - Usually, one command is retrieved and executed per iteration of the run loop, though this can be overwritten (cf. documentation).
3. **Execute the custom logic inside the `run_step()` method.**
   - By default, this method is empty (as you probably already saw inside the `template` node).
4. **Timekeeping**
   - … which we will discuss now.

To keep the framework operating at a somewhat predictable, even pace, every iteration of the run loop should take approximately the same amount of time. This interval is called `dt` and defined in the `main_config`. By default, it is set to $DT = 0.1$ seconds, meaning nodes will execute their run loop ten times a second.

If the execution of the code inside the run-loop finishes sooner than $DT$, the node waits out the remainder of the interval. If it takes longer than $DT$, you will get a warning, so you can identify which nodes are causing potential delays. (See the documentation about details on when exactly warnings are given and how you can customize this, e.g. if you know a node is frequently going to exceed $DT$.)

You probably won't need to access the internal workings of the run loop very often. Most of the time, any code that you want to be executed once per loop iteration, should be added to the `run_step()` function. (A few exceptions are discussed in a note at the end.)

**Your Task:**

First, we will continue to only publish messages when a key is pressed, but we want to update the subscriber node so it checks for new messages automatically, using `run_step()` and the `print_topic()` message you wrote in Task 2.3.

- Either copy over the code from your `print_topic()` method into the `run_step()` method or directly call that function inside `run_step()`.
- Run the code and press the key to publish a message to your topic. The subscriber should retrieve and print it almost instantaneously.

Next, we also want to automate the message publishing.

- Move (a call to) the publisher's function from Task 2.2 into the publishing node's `run_step()`.
- If you don't want to be overwhelmed by new messages being printed ten times a second, you can either change the publisher back to only retrieving and printing new messages when triggered by a keystroke or you can use the node's built-in `self.count` attribute:

```
if self.count % 10 == 0:
  self.read_topic(<your parameters>)
```

## Task 2.5: Multi-Dimensional Messages

Finally, we want to try publishing some more complex messages. So far, every message has consisted of a simple list containing single values, e.g. for the robot's pose:

```
#      time,   x,   y
msg = (100.0, 0.1, 2.34)
```

However, sometime topics require message fields that can handle multiple dimensions, i.e. lists of values.

The actual `robot_pose` topic as implemented in the current framework, for instance, has as its final field the robot's error, which tells us how certain the robot is about its current position (you will learn all about this in the context of SLAM later this term). The error is defined using a list of three values, which means that messages published to this topic actually look like this :

```
#      time,   x,   y,   theta,     error
msg = (100.0, 0.1, 2.34, 0.01, [0.02, 0.01, 0.0])
```

i.e. the final message field contains a list.

Of course, you could give each of the three values its own message field, but this would quickly become difficult when you are dealing with larger multidimensional data. For example, the images captured by the camera are stored as a nested array of integers with dimensions `480 x 864 x 3`, representing `image_height x image_width x RGB-channels`. Setting a message field for each single value in an image would not be feasible, so instead, we want the image to be stored as a single element inside the `camera_capture` message.

To define a message field as a multi-dimensional array, you need to define its dimensions in the `topic_config`:

```
robot_pose:
  buffer_length: min
  dtypes:
    names: ["timestamp", "x", "y", "theta", "error"] # error: three values for
error elipse
    formats: ["int", "float64", "float64", "float64", ["float64", [3]]]
```

As you can see, the final element in the list of formats is not a string but `["float64", [3]]`. This means that this field is an array containing three values of type `float`.

The config for the `camera_capture` topic, looks like this:

```
camera_capture:
  buffer_length: short # covers 2-3 seconds
  dtypes:
    names: ["timestamp", "img", "cam_fps"]
    formats: ["int", ["uint8", [480, 864, 3]], "int"]
```

Here, each element in the `img` field is a nested array with dimensions `(480 x 864 x 3)`.

*(**Note:** If you look at `topic_config.yaml`, you will notice that the actual `camera_capture` message has placeholders for `im_width` and `im_height`. These are defined in the main config and get replaced by the `config_parser` during start-up.)*

The data type specified for a multi-dimensional message field is intended to describe every element in that array. It is your responsibility to make sure that this is the case! If you set

`casting` to `False` and try to publish an array of `int`s into a message field of data type `float`, you will get an error. However, if your array holds a mix of ints and floats, they will not be checked individually but will all be cast to `float` during publishing, so keep that in mind.

There is one more level of complexity to consider when it comes to multi-dimensional messages. As mentioned above, messages that get written to the same topic always need to have exactly the same length. This is not a problem for topics like `robot_pose` or `camera_capture`. The robot pose always consists of singular values `x`, `y`, `theta` and a three-element list `error`. Likewise, every camera image has set dimensions that don't change over time.

However, you will often come across situations where you want to publish lists of variable length to a topic, e.g. when publishing the result of your vision node's marker detection. For any given camera image that contains detectable landmarks, the vision node might find anywhere from one to a couple dozen markers. The `marker_detection` topic, which receives the outcome of the vision node's detection method, currently has (among others) the following fields:

```
dtypes: # truncated, cf. topic_config.yaml for full list
    names: ["count",   "ids",       "landmark_rs",     "landmark_positions"]
    formats: ["int", ["int", [50]], ["float32", [50]], ["float32", [50, 2]]]
```

You can see that only `count` is a simple variable, every other field is a list of values - or even a list of lists in the case of `landmark_positions` - and most of these lists have length 50.

If your vision node only detects, say, three markers, it cannot publish a list of the three marker IDs directly to the message's `ids` field. The length of a message always needs to be the same and the length defined in the config is 50, so a list of three would be too short and raise an error.

To account for this, the `publish_to_topic()` method has the parameter `pad_arrays` that we glossed over earlier. `pad_arrays` is `False` by default, but if you set it to `True`, the function will automatically extend any list or array with too few entries for its field to the required length before publishing it to the topic. This means you only have to consider what the maximum number of entries in the list could be and set that as the length of the list in the config.

Another example: The task description states that all ArUco markers between number 1 and 550 might be used for the final map. That means the config for `landmark_poses`, the topic which tracks the position of each marker, looks like this:

```
landmark_poses:
  buffer_length: min
  dtypes:
    names: ["timestamp", "count", "ids", "positions", "errors"]
    # count: how many landmarks are known; ids: ids of known landmarks, poisitions:
(x, y), errors: 3 values for calculating error ellipse
    formats: ["int", "int", ["int", [550]], ["float64", [550, 2]], ["float64",
[550, 3]]]
```

In reality, the final map will have fewer markers than this, but you still need to write your config with an eye toward the maximum possible number of values you might need to publish.

**An important note:** There is no easy way to determine after the fact if a message has been padded, and if so, where the padding begins. We currently use `numpy`'s `empty` padding mode, which means that e.g. in an array of `int`s the padded values could be 0 or any random number that was already stored to that particular memory cell. This means that it's your responsibility to do the necessary housekeeping to keep track of how many entries in a message field hold

actual data. For example the `marker_detection` and `landmark_poses` messages both have a field named `count`, which informs any subscribers how many IDs were actually detected/how many landmarks are currently known, i.e. how many rows of the various arrays hold meaningful data.

**Your Task:**

- Create a topic with at least one multi-dimensional message field (e.g. one field for a list of numbers and a second field for the product of these numbers) and add it to the config.
- Determine a publisher and subscriber node, and update their node configs.
- Write the necessary functions to have your publisher write data to the topic and have the subscriber read it and print it to the console.

### A Note on Buffer Length

A topic's ring buffer is a `numpy` array that lives in the CPU's working memory while the framework is running. While it might be tempting to keep as much data in the topics as possible, this could quickly overwhelm the Raspberry Pi's CPU and cause the system to fail.

So, when writing a new topic and trying to decide on the appropriate buffer length, use as much space as you need but no more. E.g. there are very few situations where you would need anything but the latest handful of messages published to `marker_detections`, so keeping that buffer short is a good idea.

Similarly, the Topic `landmark_poses` needs to be able to hold the positional data for every possible landmark, meaning it has multiple arrays of length 550. Every time the SLAM node receives new information and corrects its assumptions about the landmarks (more on this in a few weeks), it publishes one message that contains the latest information for every single landmark. So, unless you *really* want to track your SLAM node's calculations over time, there is probably never a need to look at anything but the newest message published to this topic.

On the other hand, certain implementations may require you to keep track of the robot's position over time. So, even though the robot position and landmark positions are stored in the same array inside the SLAM node, they are published to two separate topics so you can set the buffer length for the smaller `robot_pose` much higher without causing too much memory clogging.

To help you estimate the size of working memory needed for your current topics, you can navigate to `robot_code/utils` in your terminal and run `python config_utils.py`. This will print a summary of the memory requirements for your current setup (according to the `node_config` and `topic_config` files). And if you want to keep track of how many resources your robot is using while it is running, you can use a program like `htop`, which comes pre-installed on your Raspberry Pi.

### A Note on Run Loop Alterations

There are situations in which you might want to alter a node's built-in run loop. For example, the `ev3` node publishes an initial message to its topics before it goes into the normal run loop (this will be needed later for SLAM). So, it overrides the normal run loop to add this publishing

step before executing the method as implemented in the base-class:

```
def run(self):
    """
    Publishes an initial message to all topics before calling standard run().
    """
    self.update_msgs(init=True)
    super().run()
```

Another scenario where you might want to override the run loop is if you have nodes that are only needed occasionally to perform very specific functions. For example, you might want to have a dedicated node for path-planning, since this can often take a while to execute and you would not want any of the other nodes to freeze up while the planner calculates the route. At the same time, the path planner does not need to regularly read information from topics or communicate with other nodes, so there is little value in having it on a standard run loop, taking up resources.

In a case like this, you can set up the node registry so that the path planner node has only a single queue which functions as its system and command queue (instructions on how to do this can be found in the documentation [coming soon]) and you can write a run method whose while-loop only contains the following:

```
while self.running:
    msg = self.sys_queue.get()
    self.parse_command(msg)
```

This would mean the node only becomes active when it receives a command through its system queue and sleeps the rest of the time without taking up computing resources.

**Congratulations!**

You now have an understanding of the basic functionality of the GöROS framework and know how to adapt it to your own needs.

As you have probably seen by now, we have already defined some nodes and topics that will help you get started on your own implementation. You can find details on those in the documentation (coming soon), but for now, we will update the framework so you can drive your robot manually and receive images from its camera through the viewer.

For this, you need to copy the gitlab repository (or at least the `robot_code` folder) to your Raspberry Pi. There, you only need to update the `node_config.yaml` file by including three nodes:

```
---
include:
  # SYSTEM NODES (always required)
  - main # main node always needs to be included!
  - logger # logging node, will be included if logging is enabled
  # CUSTOM NODES
  - robot_controller
  - ev3
  - camera
  # Add your own nodes below
```

The configs and classes for these nodes are already provided for you, as well as the key press dictionary for manual driving and the topics needed for the camera to communicate with the viewer.

Now, when you start up the framework on your Raspberry Pi, you can switch to manual mode by pressing `m` and then drive using `WASD` (and stop driving by pressing `c`).

To see the images transmitted from your robot's camera, you need to make sure the IP address in `subscriber.py` on your machine matches the robot's IP (which you can check with `> tailscale status`):

```
class Subscriber():
    def __init__(self, port='tcp://100.64.0.1:5555') -> None:
```

(`5555` is the number of the port that the robot and the viewer use to communicate.)

Once that is done, you can start the robot by running `python main.py` on your Raspberry Pi and `python viewer.py` on your local machine and you should be able to see the camera transmission in real time.

**Happy Roboting!**